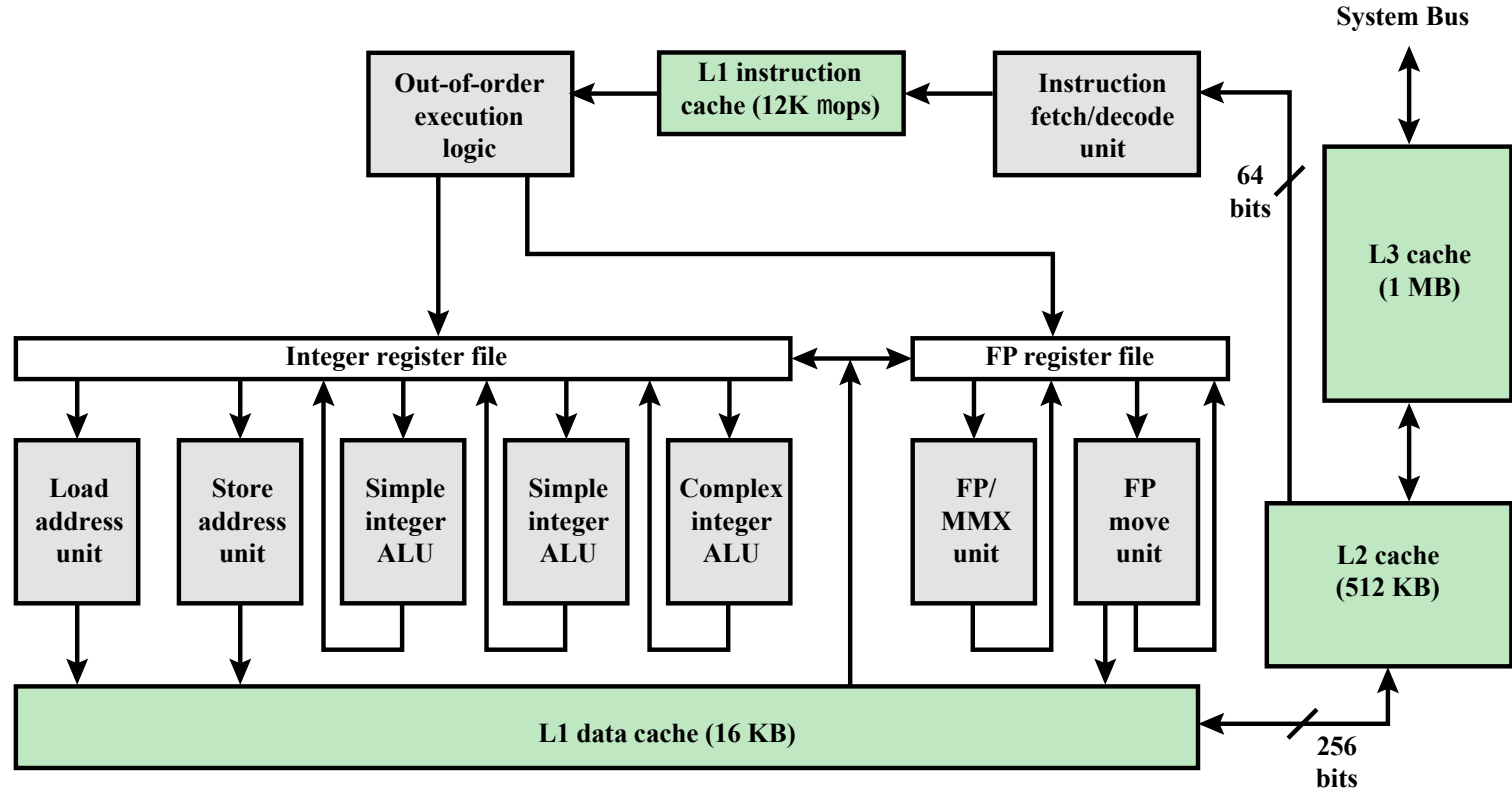
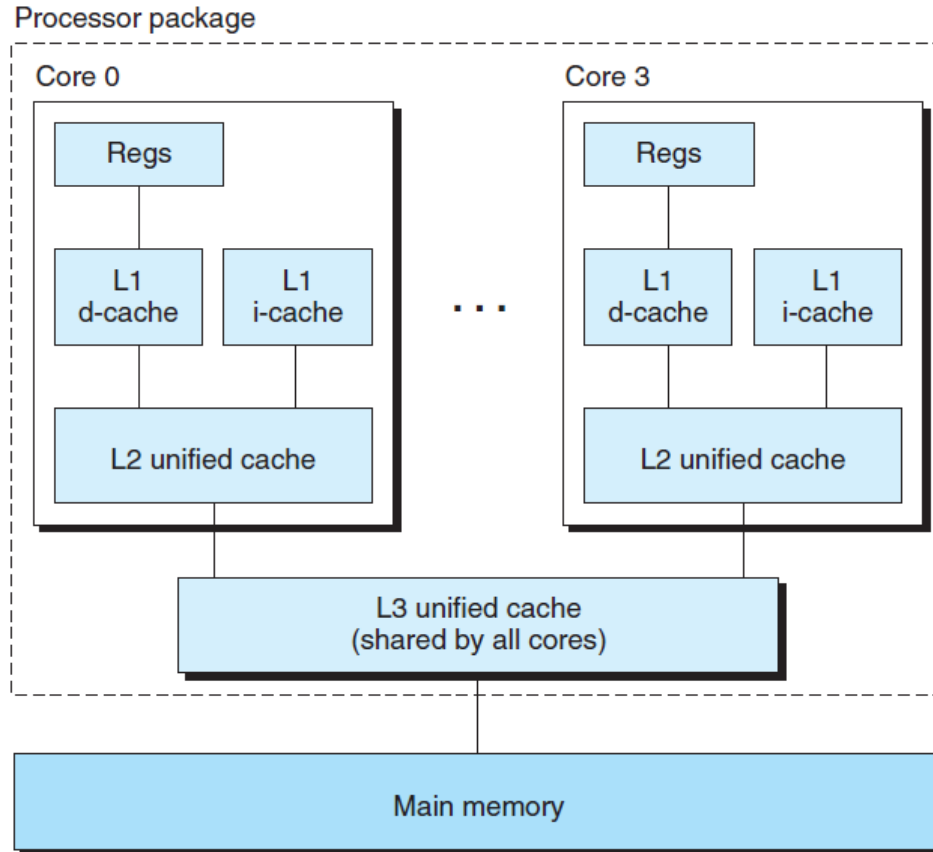


Example architecture: Pentium 4

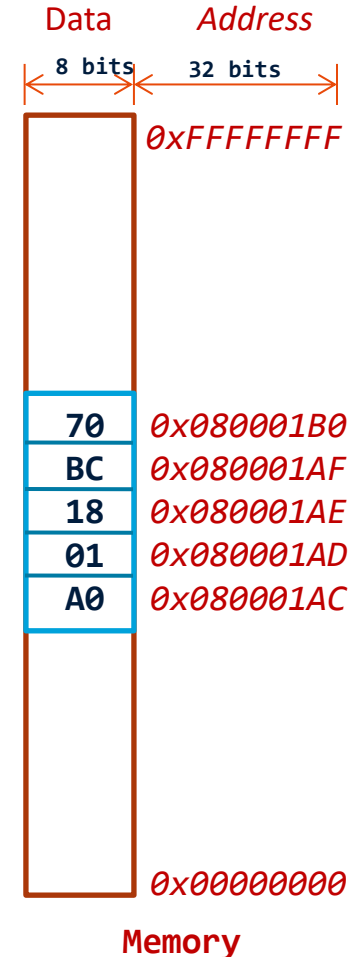


Example architecture: Core I7



Memory

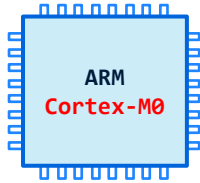
- Memory is arranged as a series of “locations”
 - Each location has a unique “address”
 - Each location holds a byte (**byte-addressable**)
 - e.g. the memory location at address 0x080001B0 contains the byte value 0x70, i.e., 112
- The number of locations in memory is limited
- Values stored at each location can represent either **data** or **instructions**
 - e.g. the value 0x70 might be the code used to tell the processor to add two values together



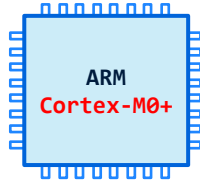
ARM Cortex-M Series Family

Von-Neumann

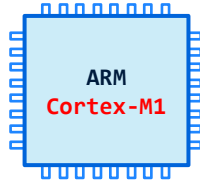
Instructions and data are stored in the same memory.



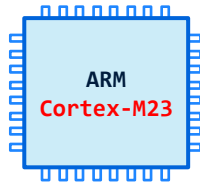
ARMv6-M



ARMv6-M



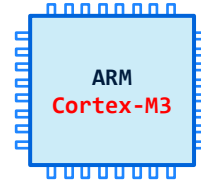
ARMv6-M



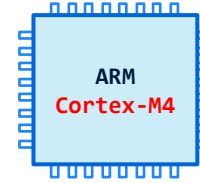
ARMv8-M

Harvard

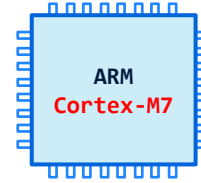
Data and instructions are stored into separate memories.



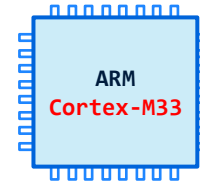
ARMv7-M



ARMv7E-M



ARMv7E-M

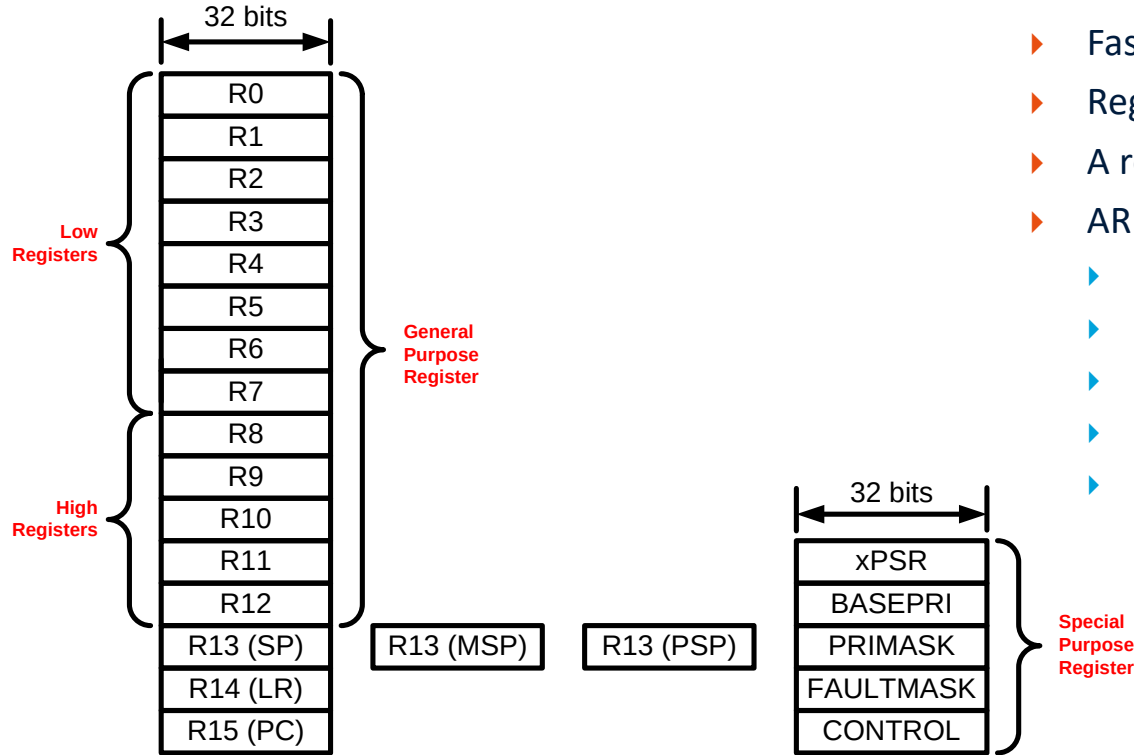


ARMv8-M



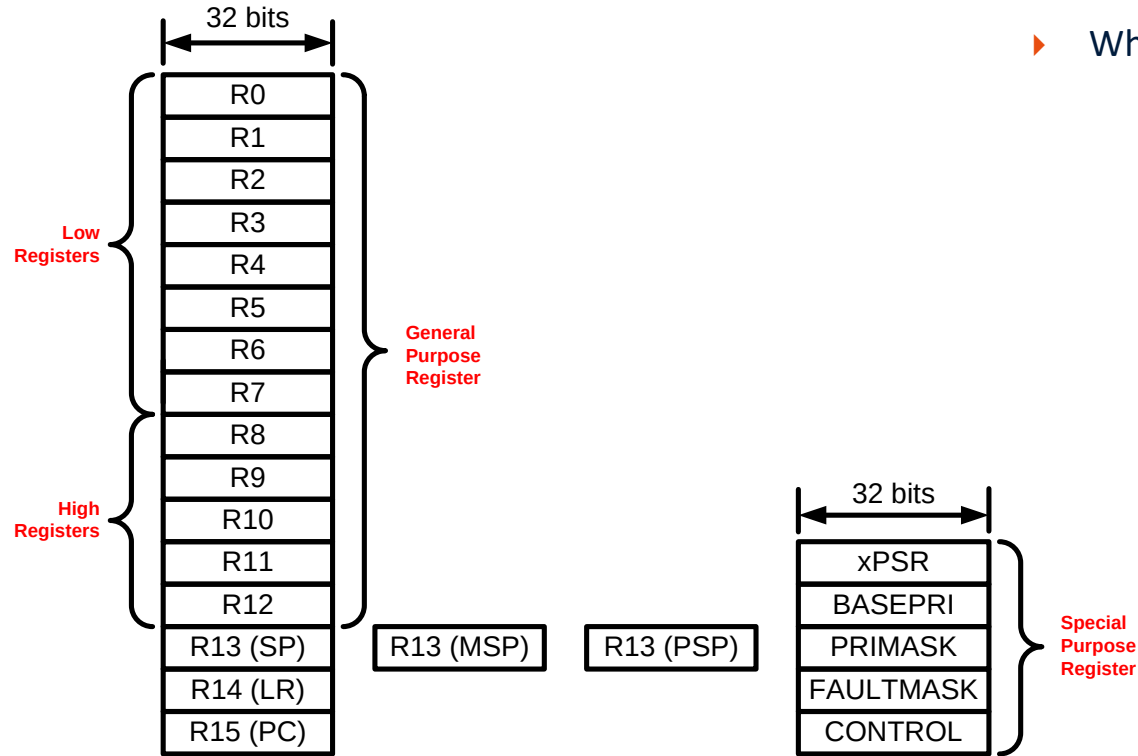
- 5

Processor Registers



- ▶ Fastest way to read and write
- ▶ Registers are within the processor chip
- ▶ A register stores a 32-bit value
- ▶ ARM Cortex-M has
 - ▶ **R0-R12:** 13 general-purpose registers
 - ▶ **R13:** Stack pointer (Shadow of MSP or PSP)
 - ▶ **R14:** Link register (LR)
 - ▶ **R15:** Program counter (PC)
 - ▶ Special registers (xPSR, BASEPRI, PRIMASK, etc)

Processor Registers

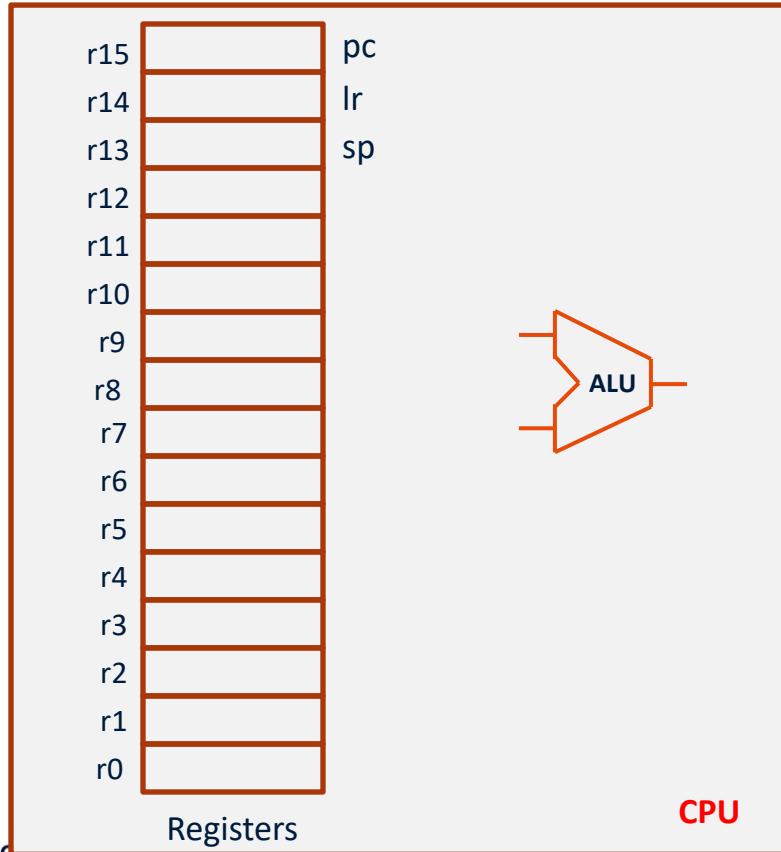


Processor Registers

Machine	Number of general-purpose registers	Architectural style	Year
EDSAC	1	Accumulator	1949
IBM 701	1	Accumulator	1953
CDC 6600	8	Load-store	1963
IBM 360	16	Register-memory	1964
DEC PDP-8	1	Accumulator	1965
DEC PDP-11	8	Register-memory	1970
Intel 8008	1	Accumulator	1972
Motorola 6800	2	Accumulator	1974
DEC VAX	16	Register-memory, memory-memory	1977
Intel 8086	1	Extended accumulator	1978
Motorola 68000	16	Register-memory	1980
Intel 80386	8	Register-memory	1985
ARM	16	Load-store	1985
MIPS	32	Load-store	1985
HP PA-RISC	32	Load-store	1986
SPARC	32	Load-store	1987
PowerPC	32	Load-store	1992
DEC Alpha	32	Load-store	1992
HP/Intel IA-64	128	Load-store	2001
AMD64 (EMT64)	16	Register-memory	2003
ARMv8	16	Load-store	2013

- ▶ Why only 16 general purpose registers?
- ▶ **Design Principle: Smaller is faster**
- ▶ A very large number of registers may increase the clock cycle time simply because it takes electronic signals longer when they must travel farther
- ▶ Guidelines such as “smaller is faster” are not absolutes; 15 registers may not be faster than 16. Even so, the truth behind such observations causes computer designers to take them seriously. In this case, the designer must balance the craving of programs for more registers with the desire to keep the clock cycle fast

Machine codes are stored in memory

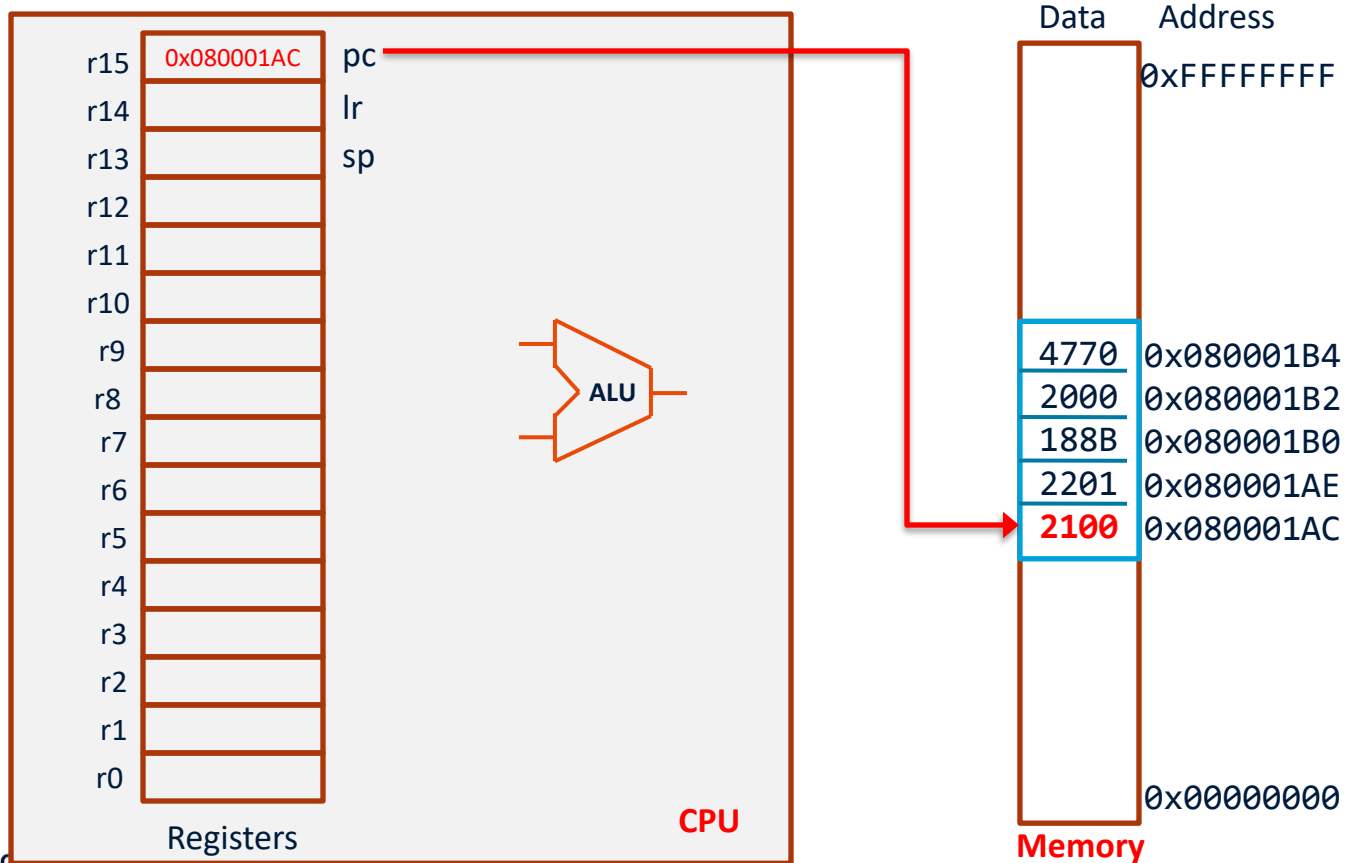


Data	Address
	0xFFFFFFFF
4770	0x080001B4
2000	0x080001B2
188B	0x080001B0
2201	0x080001AE
2100	0x080001AC
	0x00000000

Memory

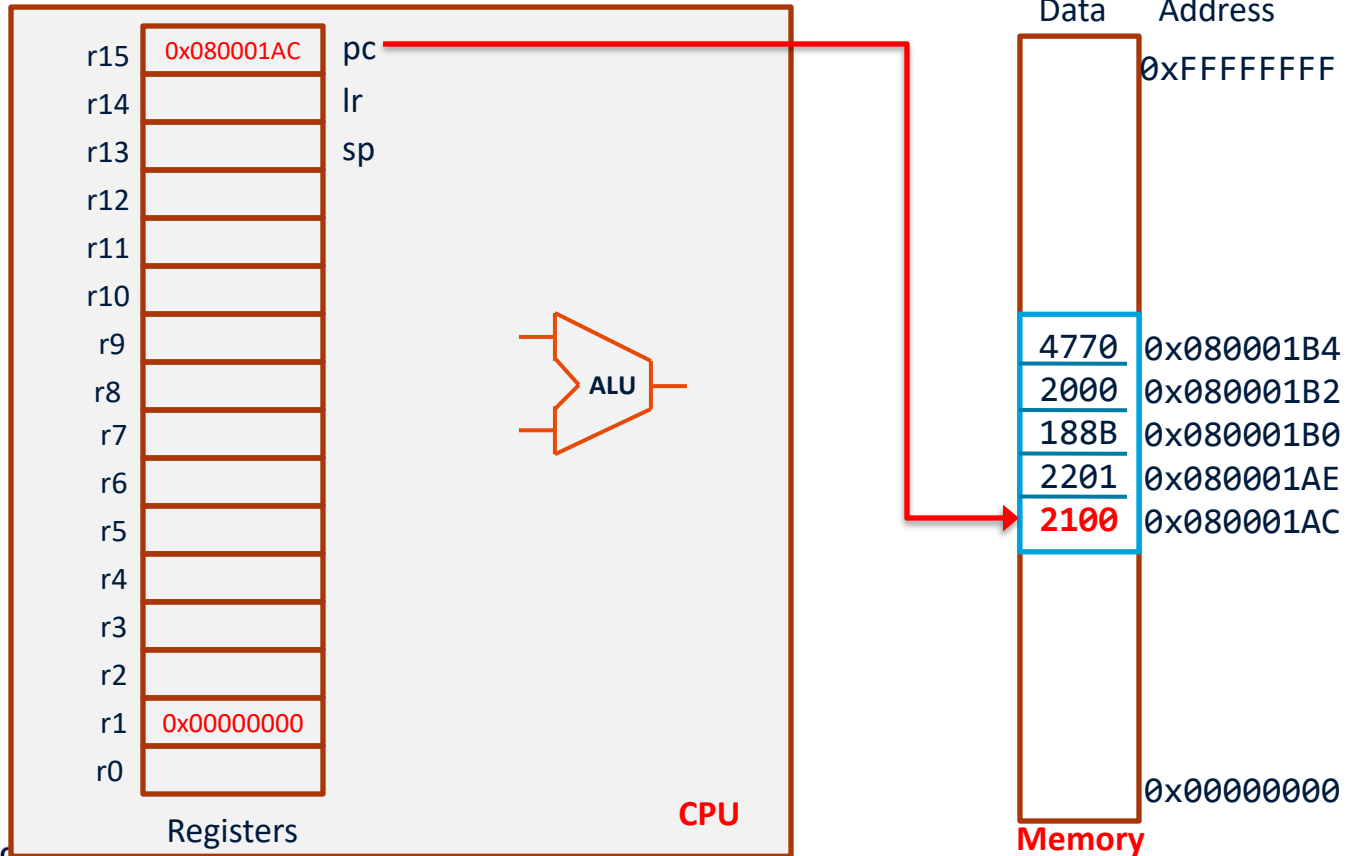
Fetch Instruction: pc = 0x08001AC

Decode Instruction: 2100 = MOVs r1, #0x00



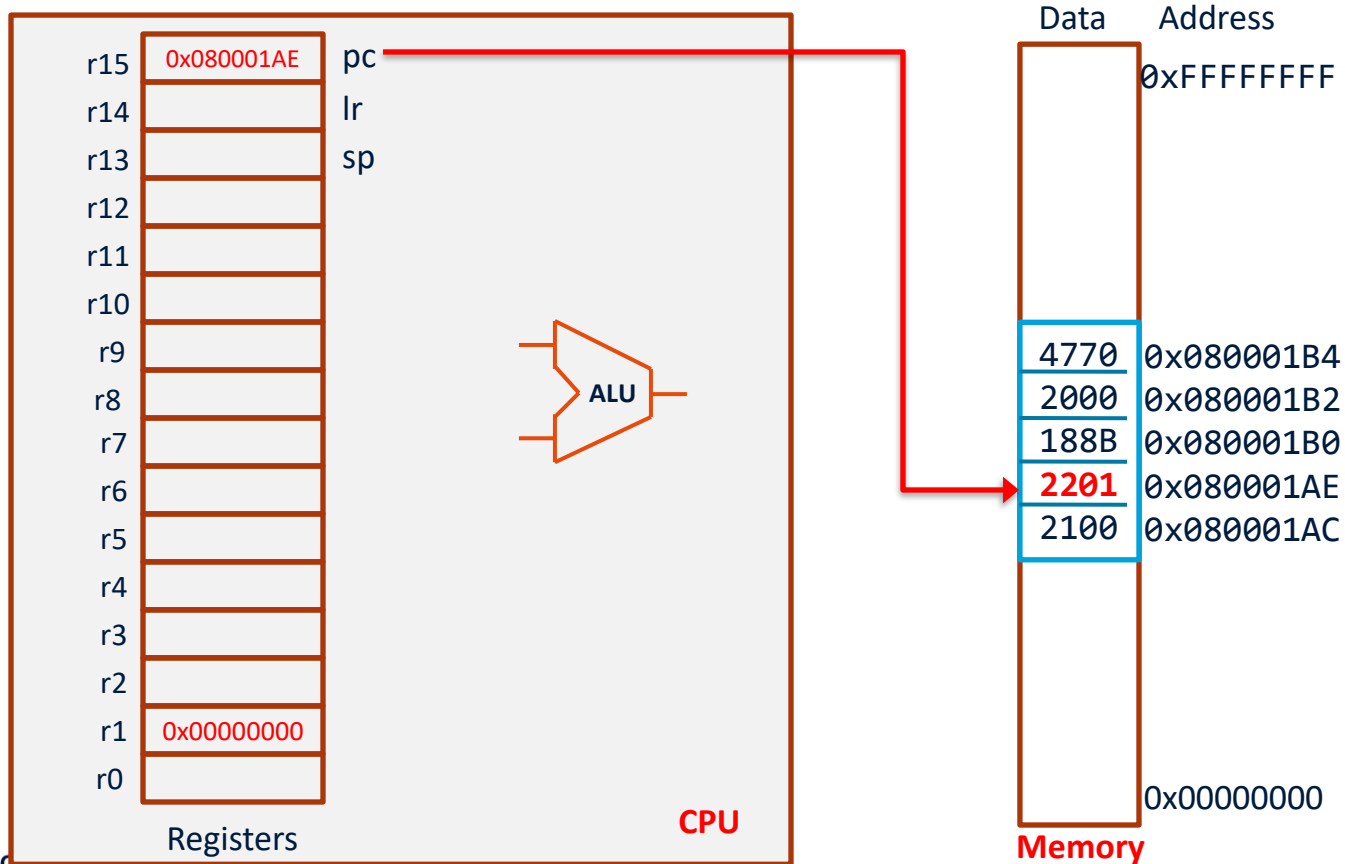
Execute Instruction:

MOVS r1, #0x00



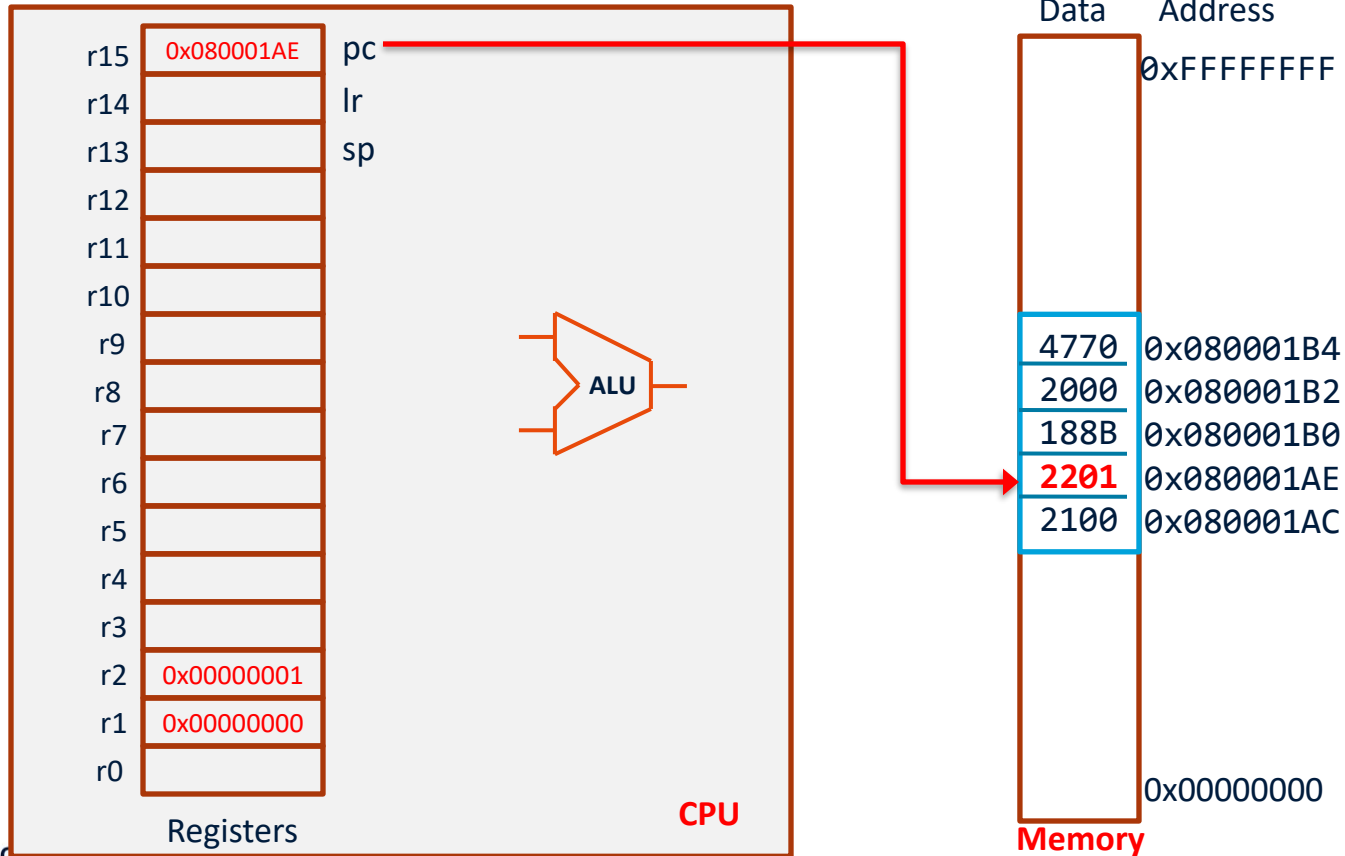
Fetch Next Instruction: $pc = pc + 2$

- Thumb-2 consists of a mix of 16- & 32-bit instructions
- In reality, we always fetch 4 bytes from the instruction memory (either one 32-bit instruction or two 16-bit instructions)
- To simplify the demo, we assume we only fetch 2 bytes from the instruction memory in this example.



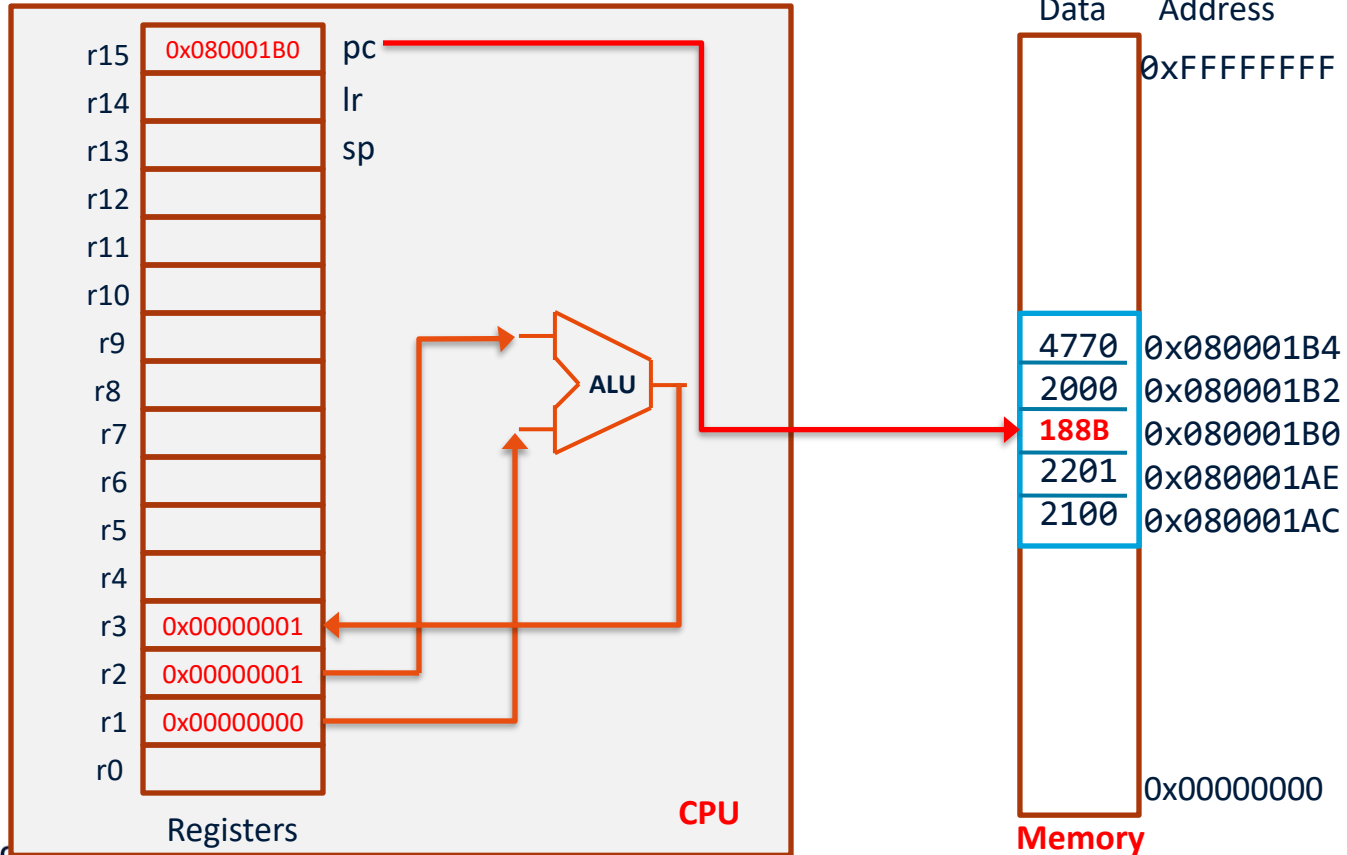
Fetch Next Instruction: $pc = pc + 2$

Decode & Execute: $2201 = \text{MOVS } r2, \#0x01$



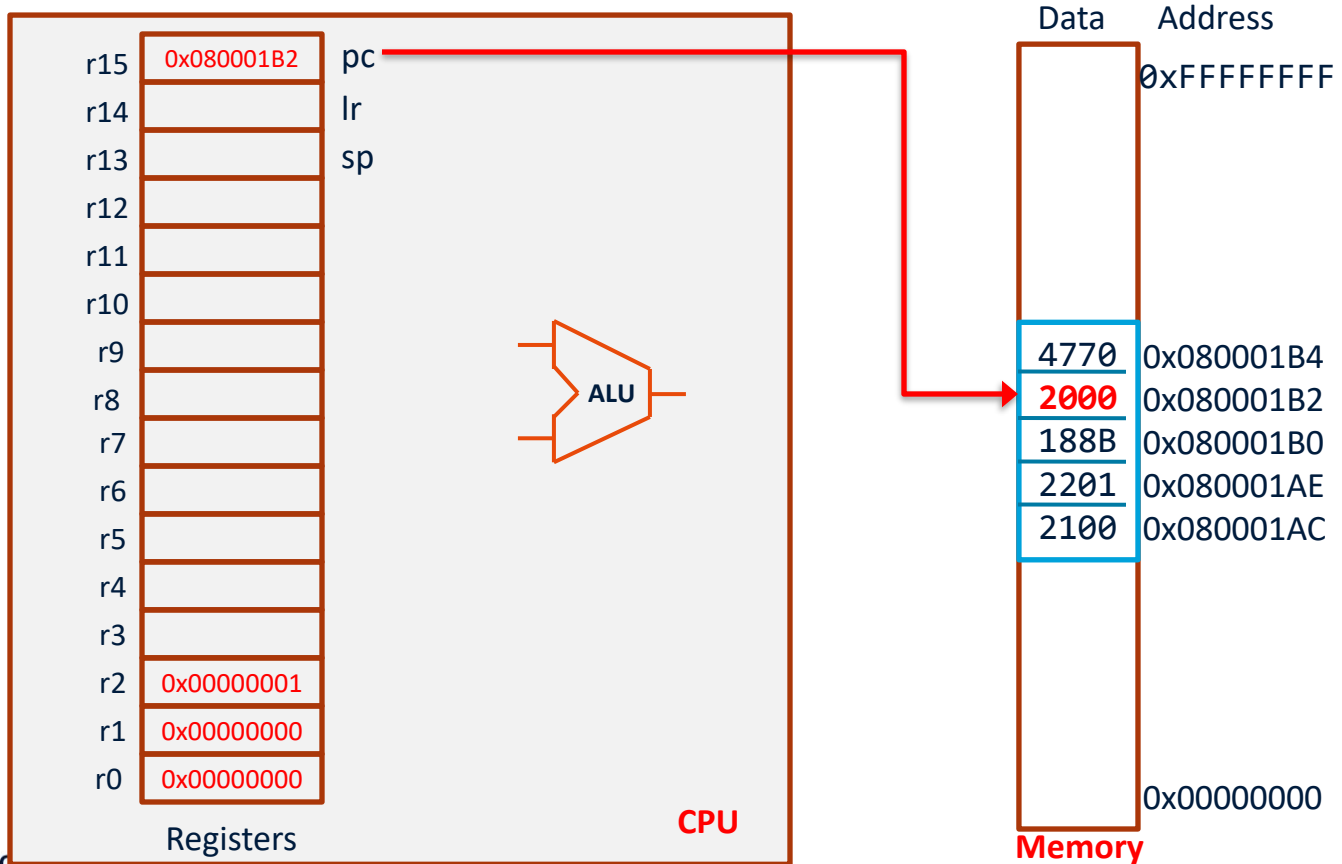
Fetch Next Instruction: $pc = pc + 2$

Decode & Execute: $188B = \text{ADDS } r3, r1, r2$



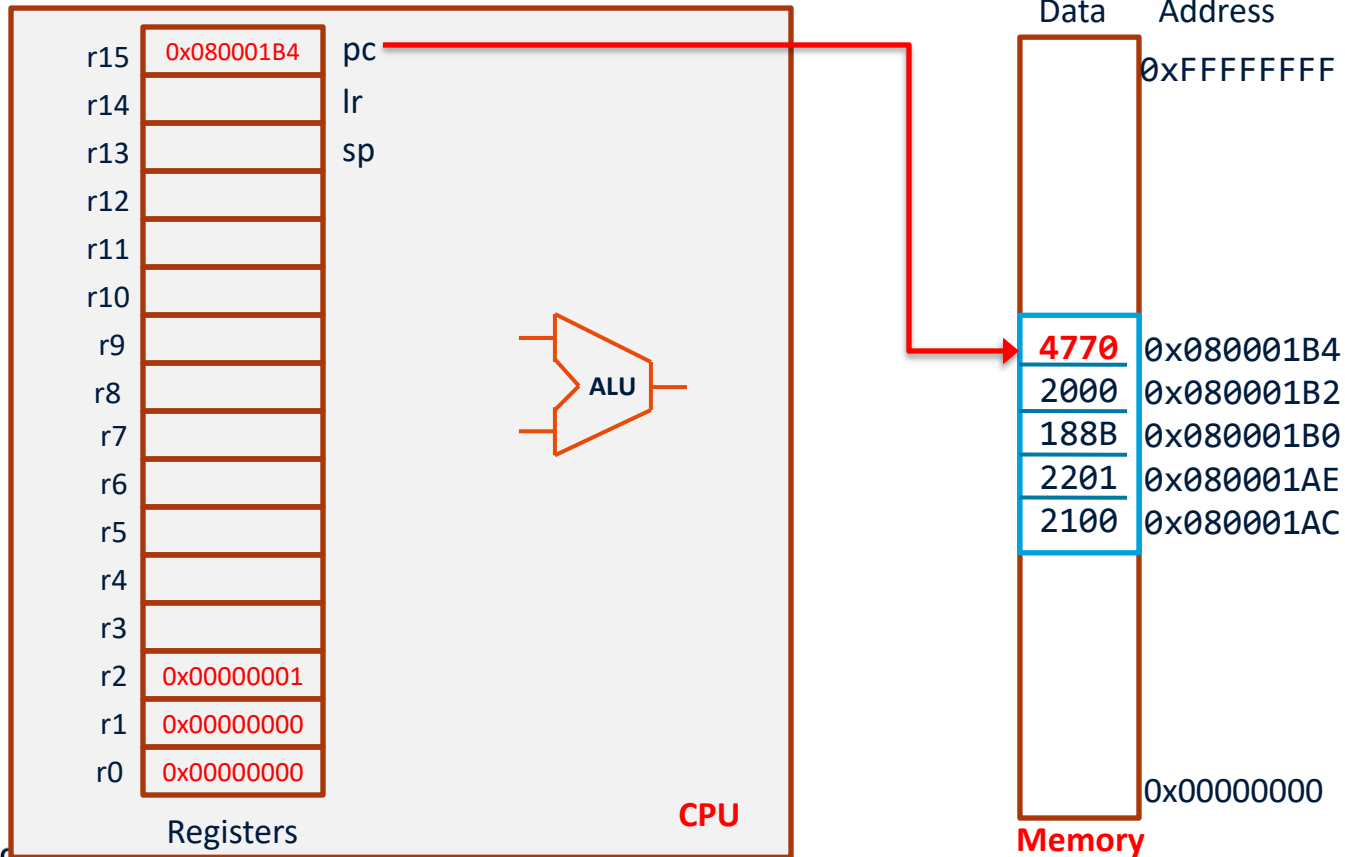
Fetch Next Instruction: $pc = pc + 2$

Decode & Execute: $2000 = \text{MOVS } r0, \#0x00$



Fetch Next Instruction: $pc = pc + 2$

Decode & Decode: $4770 = \text{BX } lr$



Example:

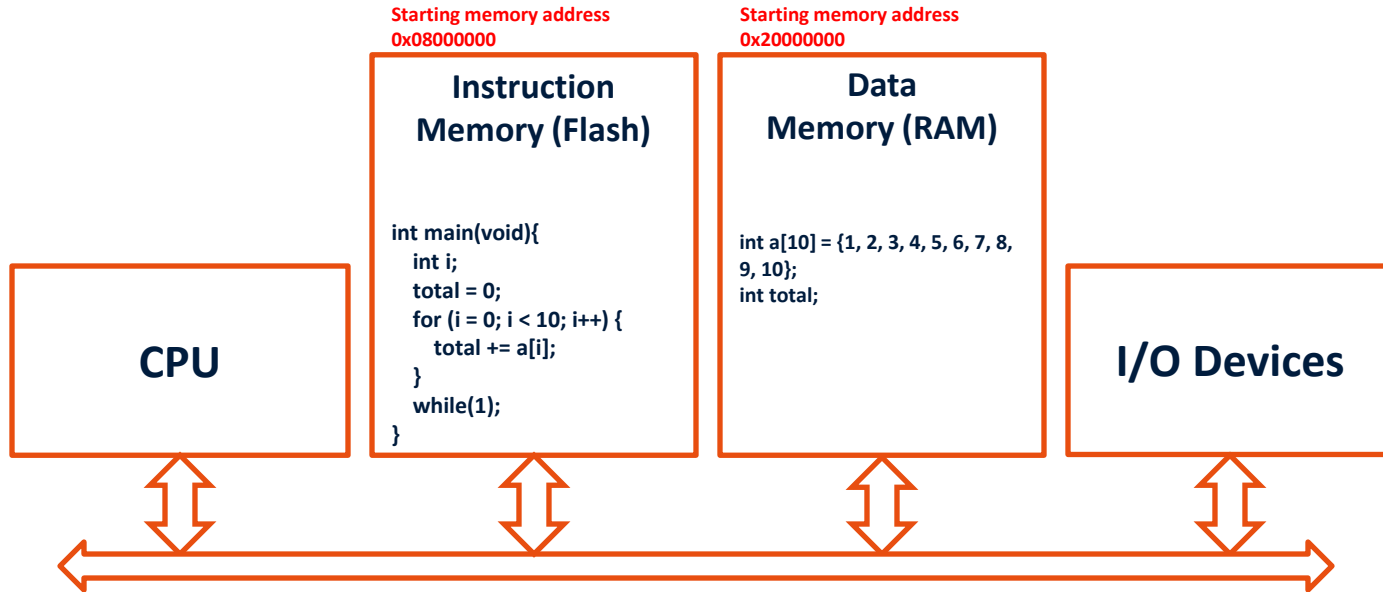
Calculate the Sum of an Array

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int total;

int main(void){
    int i;
    total = 0;
    for (i = 0; i < 10; i++) {
        total += a[i];
    }
    while(1);
}
```

Example:

Calculate the Sum of an Array



Example:

Calculate the Sum of an Array

Instruction Memory (Flash)

```
int main(void){  
    int i;  
    total = 0;  
    for (i = 0; i < 10; i++) {  
        total += a[i];  
    }  
    while(1);  
}
```

Starting memory address
0x08000000

0010 0001 0000 0000
0100 1010 0000 1000
0110 0000 0001 0001
0010 0000 0000 0000
1110 0000 0000 1000
0100 1001 0000 0111
1111 1000 0101 0001
0001 0000 0010 0000
0100 1010 0000 0100
0110 1000 0001 0010
0100 0100 0001 0001
0100 1010 0000 0011
0110 0000 0001 0001
0001 1100 0100 0000
0010 1000 0000 1010
1101 1011 1111 0100
1011 1111 0000 0000
1110 0111 1111 1110



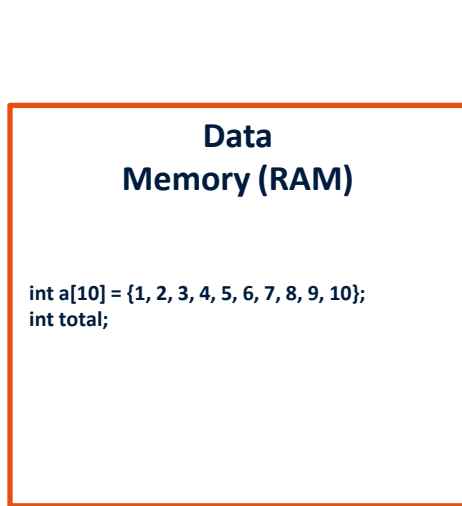
```
12  
0800027e: 20 4b  
08000280: 00 22  
08000282: 1a 60  
13  
08000284: 00 23  
08000286: 7b 62  
08000288: 0b e0  
14  
0800028a: 1e 4a  
0800028c: 7b 6a  
0800028e: 52 f8 23 20  
08000292: 1b 4b  
08000294: 1b 68  
08000296: 13 44  
08000298: 19 4a  
0800029a: 13 60  
13  
0800029c: 7b 6a  
0800029e: 01 33  
080002a0: 7b 62  
080002a2: 7b 6a  
080002a4: 09 2b  
080002a6: f0 dd
```

```
    total = 0;  
    ldr    r3, [pc, #128] @ (0x8000300 <main+136>  
    movs   r2, #0  
    str    r2, [r3, #0]  
    for (i = 0; i < 10; i++) {  
    movs   r3, #0  
    str    r3, [r7, #36] @ 0x24  
    b.n    0x80002a2 <main+42>  
        total += a[i];  
    ldr    r2, [pc, #120] @ (0x8000304 <main+140>  
    ldr    r3, [r7, #36] @ 0x24  
    ldr.w  r2, [r2, r3, lsl #2]  
    ldr    r3, [pc, #108] @ (0x8000300 <main+136>  
    ldr    r3, [r3, #0]  
    add    r3, r2  
    ldr    r2, [pc, #100] @ (0x8000300 <main+136>  
    str    r3, [r2, #0]  
    for (i = 0; i < 10; i++) {  
    ldr    r3, [r7, #36] @ 0x24  
    adds  r3, #1  
    str    r3, [r7, #36] @ 0x24  
    ldr    r3, [r7, #36] @ 0x24  
    cmp    r3, #9  
    ble.n  0x800028a <main+18>
```

Example:

Calculate the Sum of an Array

Address	0	2	4	6	8	A	C	E
20000000	0100	0000	0200	0000	0300	0000	0400	0000
20000010	0500	0000	0600	0000	0700	0000	0800	0000
20000020	0900	0000	0A00	0000	0000	0000	0000	0000
20000030	0000	0000	0000	0000	0000	0000	0000	0000



Assume the starting memory address of the data memory is 0x20000000

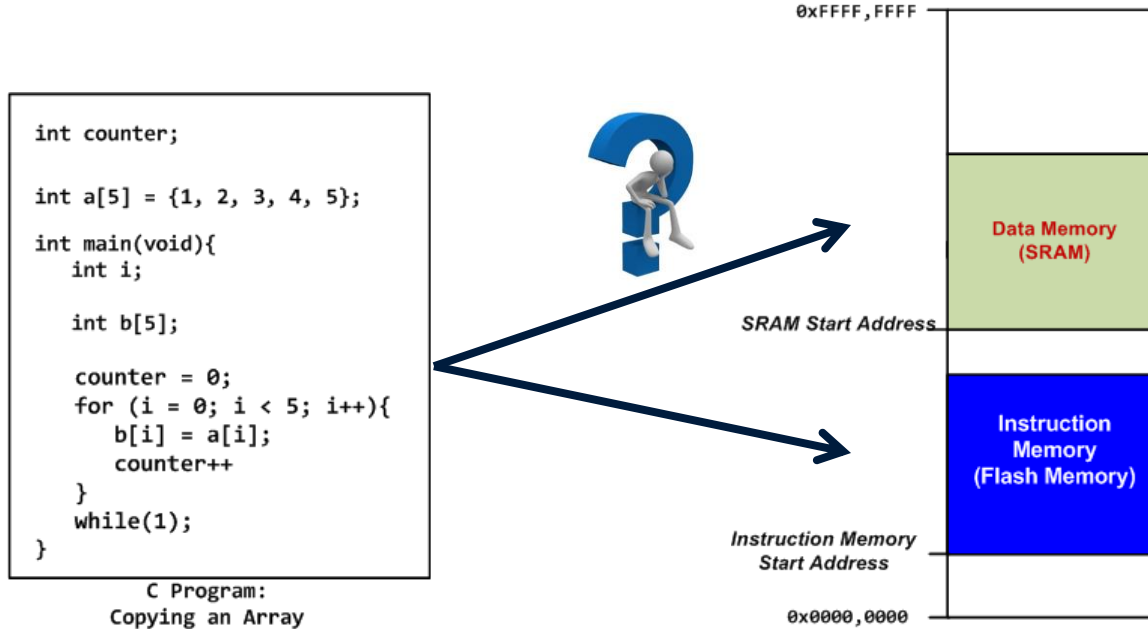
0x20000054	0x00000000
0x20000050	0x00000000
0x2000004C	0x00000000
0x20000048	0x00000000
0x20000044	0x00000000
0x20000040	0x00000000
0x2000003C	0x00000000
0x20000038	0x00000000
0x20000034	0x00000000
0x20000030	0x00000000
0x2000002C	0x00000000
0x20000028	0x00000000
0x20000024	0x0000000A
0x20000020	0x00000009
0x2000001C	0x00000008
0x20000018	0x00000007
0x20000014	0x00000006
0x20000010	0x00000005
0x2000000C	0x00000004
0x20000008	0x00000003
0x20000004	0x00000002
0x20000000	0x00000001

Memory
address
in bytes

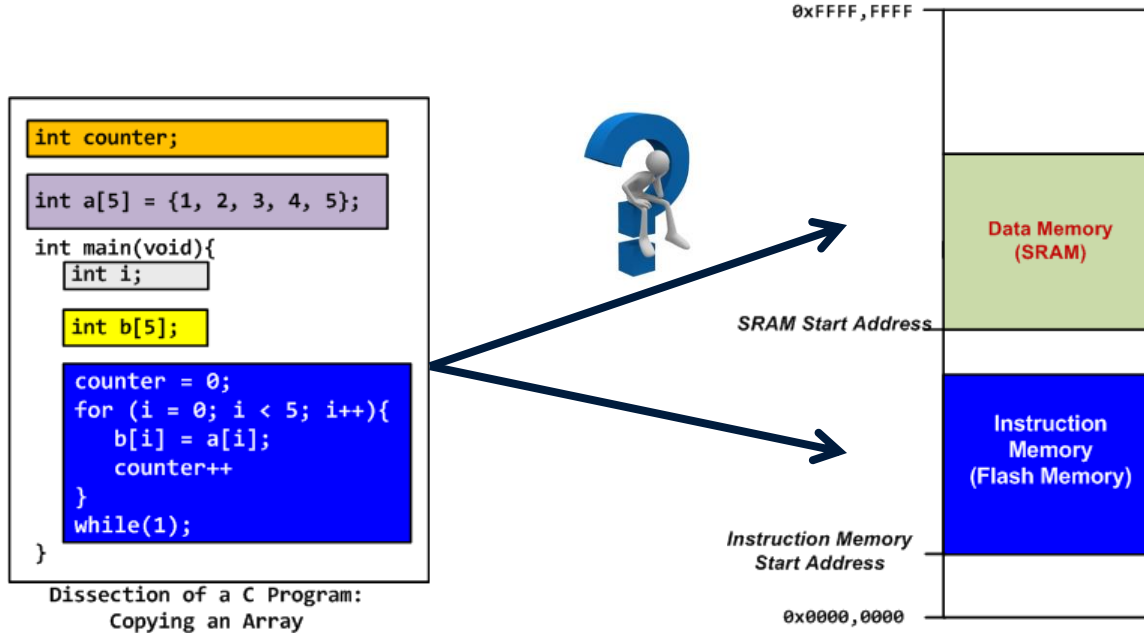
Memory
content

```
total= 0x00000000
a[9] = 0x0000000A
a[8] = 0x00000009
a[7] = 0x00000008
a[6] = 0x00000007
a[5] = 0x00000006
a[4] = 0x00000005
a[3] = 0x00000004
a[2] = 0x00000003
a[1] = 0x00000002
a[0] = 0x00000001
```

Loading Code and Data into Memory



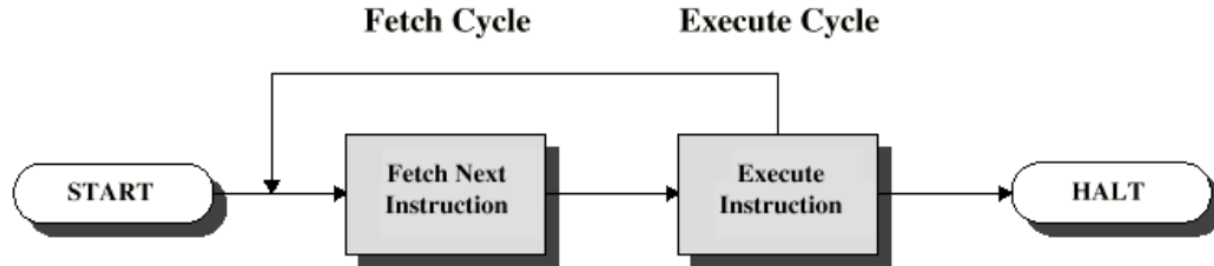
Loading Code and Data into Memory



Instruction cycle and pipelines

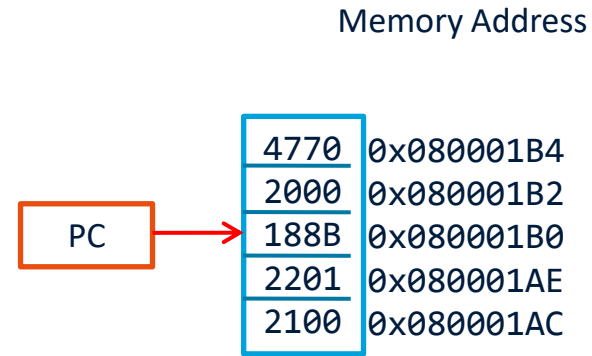
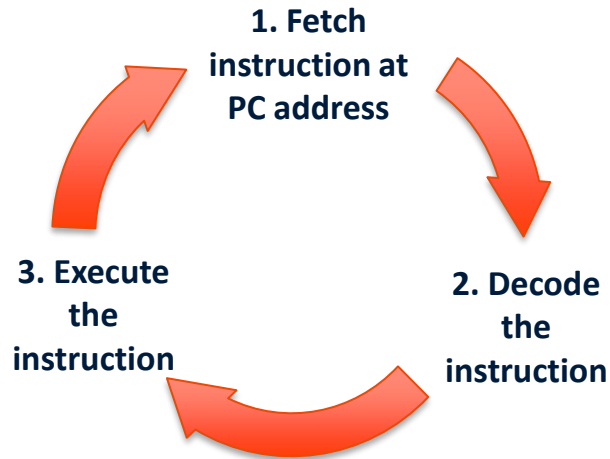
Instruction cycle

- Three Steps:
 - Fetch
 - Decode
 - Execute



Program Execution

- **Program Counter (PC)** is a register that holds the memory address of the next instruction to be fetched from the memory

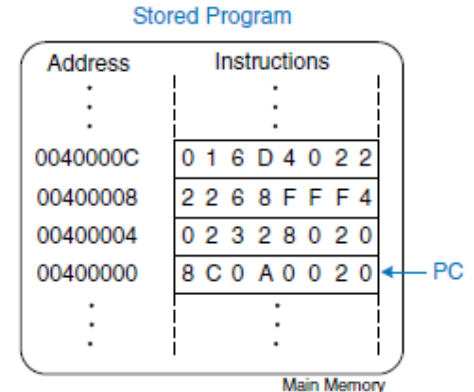


PC = 0x080001B0
Instruction = 188B or
2000188B or 8B180020

Program Execution

- To run or execute the stored program, the processor fetches the instructions from memory sequentially. The fetched instructions are then decoded and executed by the digital hardware
- To execute the code in Figure 6.13, the operating system sets the PC to address 0x00400000. The processor reads the instruction at that memory address and executes the instruction, 0x8C0A0020. The processor then increments the PC by 4, to 0x00400004

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022



Pipelines

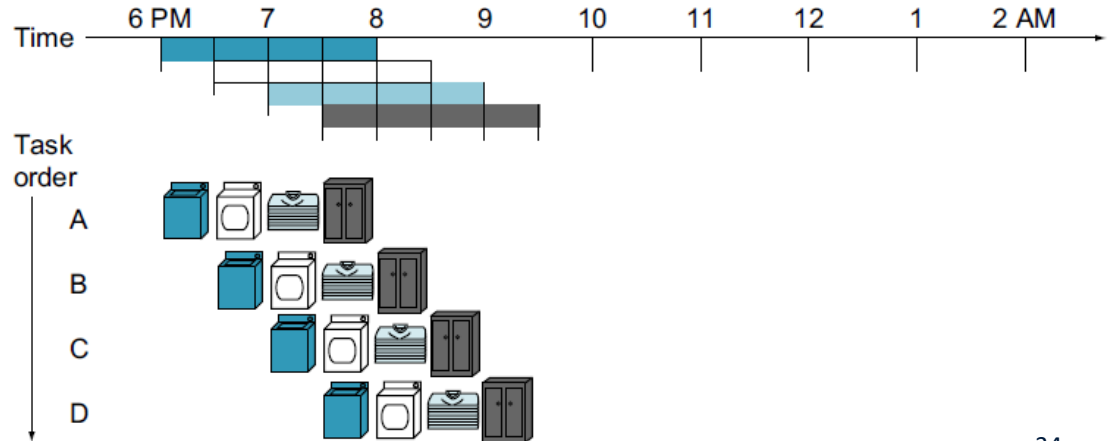
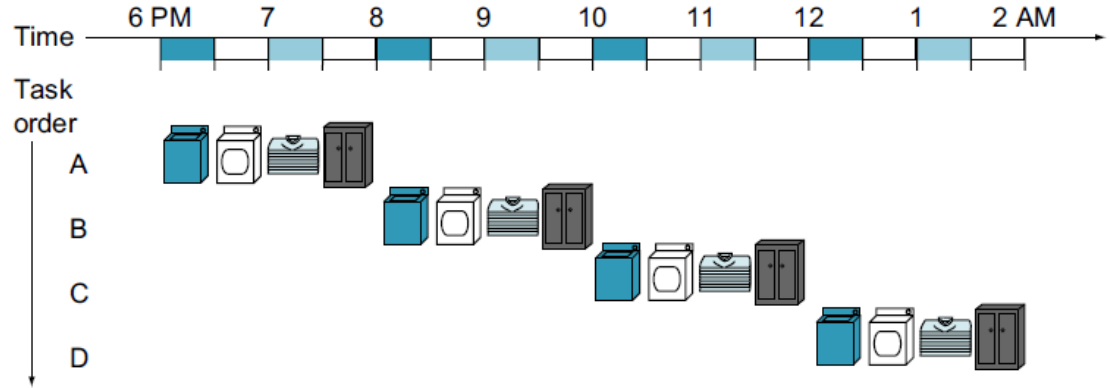
Pipelining is an implementation technique in which multiple instructions are overlapped in execution. Today, pipelining is nearly universal. Anyone who has done a lot of laundry has intuitively used pipelining. The nonpipelined approach to laundry would be as follows:

1. Place one dirty load of clothes in the washer.
2. When the washer is finished, place the wet load in the dryer.
3. When the dryer is finished, place the dry load on a table and fold.
4. When folding is finished, ask your roommate to put the clothes away



Pipelines

The pipelined approach takes much less time:

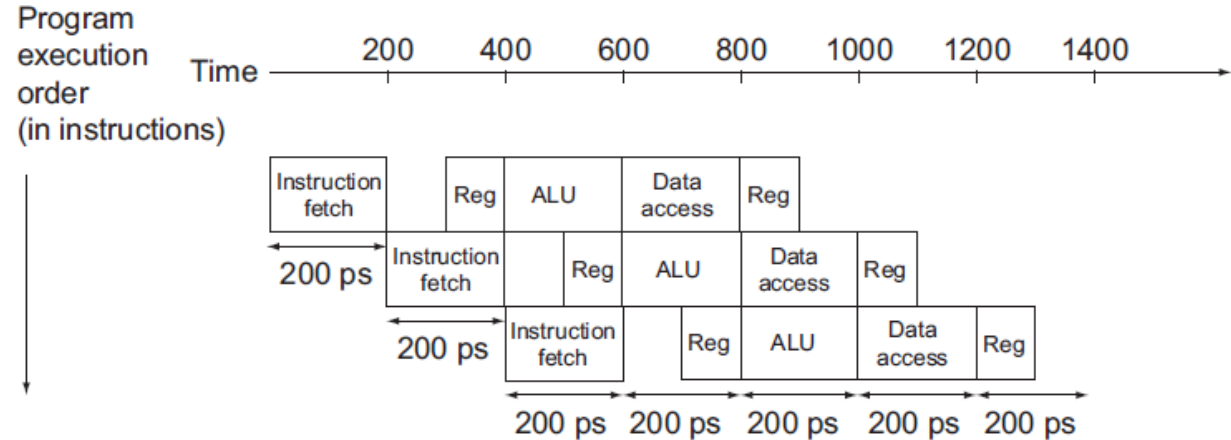
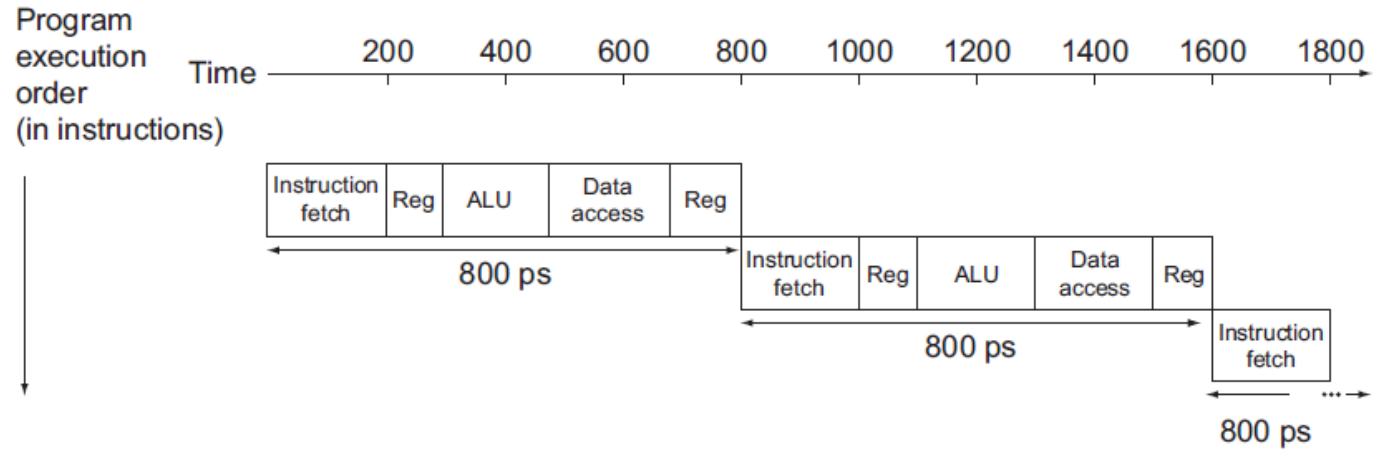


Pipelines

ARM instructions classically take five steps:

1. **Fetch** instruction from memory.
2. Read registers and **decode** the instruction.
3. **Execute** the operation or calculate an address.
4. Access an operand in data memory (if necessary).
5. Write the result into a register (if necessary).

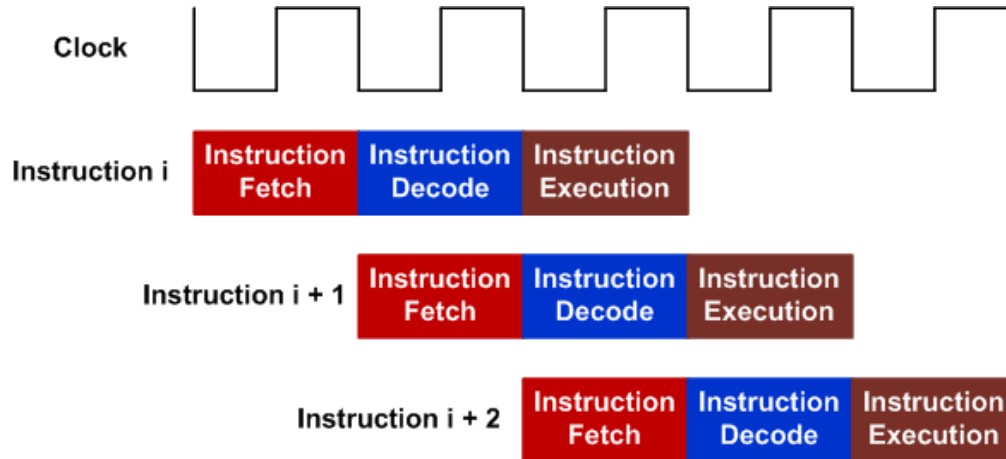
Pipelines



Three-state pipeline:

Fetch, Decode, Execution

- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.



Takeaways

- The Von Neumann and Harvard architectures
- The CPU: control, ALU and registers
- ALU: half and full adders, subtraction
- Memory types and access methods
- SRAM, DRAM and Flash
- Memory model – storing data and instructions
- Memory caches
- Registers
- Execution model: fetch decode execute
- Pipelines